

Database Schema Code

AI-Based Examination Platform

Part I	PostgreSQL DDL, compiled from the live SQLAlchemy metadata — 306 lines
Part II	Declarative model sources, verbatim — 12 files, 963 lines
Part III	Alembic migrations, verbatim — 3 files, 409 lines
Coverage	13 tables · 10 native enum types

I	Generated PostgreSQL DDL	models/question.py
II	Declarative model sources	models/exam.py
	base.py	models/exam_session.py
	session.py	models/proctor.py
	models/__init__.py	models/grading.py
	models/enums.py	III Alembic migrations
	models/user.py	03ad33e6bd09_initial_schema.py
	models/login_access.py	a17b4c9de204_word_bounds_thumbnails_ocr.py
	models/enrollment.py	f0c6eeb2272c_login_access_requests_exam_enrollments_.

Generated PostgreSQL DDL

Compiled from the live metadata with CreateTable / CreateIndex against the PostgreSQL dialect, so this is exactly what the models ask the database for. Tables appear in dependency order; constraint names come from the convention in base.py. Enum types are emitted first because every table that uses one depends on it.

```
1  -- =====
2  -- Enum types
3  -- =====
4
5  CREATE TYPE access_status AS ENUM ('pending', 'approved', 'revoked');
6  CREATE TYPE difficulty AS ENUM ('easy', 'medium', 'hard');
7  CREATE TYPE exam_status AS ENUM ('draft', 'published', 'closed');
8  CREATE TYPE grade_status AS ENUM ('unanswered', 'auto_scored', 'pending_ai', 'ai_scored', 'examiner_reviewed');
9  CREATE TYPE login_access_status AS ENUM ('pending', 'approved', 'rejected');
10 CREATE TYPE proctor_event_type AS ENUM ('face_missing', 'multiple_faces', 'gaze_away', 'tab_switch', 'window_blur',
'fullscreen_exit', 'camera_blocked', 'paste_attempt', 'copy_attempt', 'devtools_open');
11 CREATE TYPE proctor_severity AS ENUM ('info', 'warning', 'critical');
12 CREATE TYPE question_type AS ENUM ('mcq', 'multi_select', 'short_answer', 'long_answer', 'image_upload');
13 CREATE TYPE session_status AS ENUM ('in_progress', 'submitted', 'auto_submitted', 'terminated');
14 CREATE TYPE user_role AS ENUM ('admin', 'examiner', 'candidate');
15
16 -- =====
17 -- Tables, in dependency order
18 -- =====
19
20 -----
21 -- subjects
22 -----
23 CREATE TABLE subjects (
24   code VARCHAR(32) NOT NULL,
25   name VARCHAR(150) NOT NULL,
26   description TEXT,
27   id UUID NOT NULL,
28   created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
29   updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
30   CONSTRAINT pk_subjects PRIMARY KEY (id)
31 );
32 CREATE UNIQUE INDEX ix_subjects_code ON subjects (code);
33
34 -----
35 -- users
36 -----
37 CREATE TABLE users (
38   email VARCHAR(255) NOT NULL,
39   password_hash VARCHAR(255) NOT NULL,
40   first_name VARCHAR(80) NOT NULL,
41   last_name VARCHAR(80) NOT NULL,
42   full_name VARCHAR(150) NOT NULL,
43   role user_role NOT NULL,
44   is_active BOOLEAN NOT NULL,
45   access_status access_status NOT NULL,
46   access_changed_at TIMESTAMP WITH TIME ZONE,
47   access_changed_by_id UUID,
48   access_note TEXT,
49   last_login_at TIMESTAMP WITH TIME ZONE,
50   id UUID NOT NULL,
51   created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
52   updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
53   CONSTRAINT pk_users PRIMARY KEY (id),
54   CONSTRAINT fk_users_access_changed_by_id_users FOREIGN KEY(access_changed_by_id) REFERENCES users (id) ON DELETE SET
NULL
55 );
56 CREATE INDEX ix_users_access_status ON users (access_status);
57 CREATE UNIQUE INDEX ix_users_email ON users (email);
58 CREATE INDEX ix_users_role ON users (role);
59
60 -----
61 -- exams
62 -----
63 CREATE TABLE exams (
64   subject_id UUID NOT NULL,
65   title VARCHAR(200) NOT NULL,
66   description TEXT,
```

```

67 duration_minutes INTEGER NOT NULL,
68 starts_at TIMESTAMP WITH TIME ZONE NOT NULL,
69 ends_at TIMESTAMP WITH TIME ZONE NOT NULL,
70 selection_rules JSONB NOT NULL,
71 randomize BOOLEAN NOT NULL,
72 shuffle_options BOOLEAN NOT NULL,
73 negative_marking BOOLEAN NOT NULL,
74 paper_salt VARCHAR(64) NOT NULL,
75 proctor_config JSONB NOT NULL,
76 grading_config JSONB NOT NULL,
77 status exam_status NOT NULL,
78 results_published BOOLEAN NOT NULL,
79 created_by_id UUID,
80 id UUID NOT NULL,
81 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
82 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
83 CONSTRAINT pk_exams PRIMARY KEY (id),
84 CONSTRAINT fk_exams_subject_id_subjects FOREIGN KEY(subject_id) REFERENCES subjects (id) ON DELETE RESTRICT,
85 CONSTRAINT fk_exams_created_by_id_users FOREIGN KEY(created_by_id) REFERENCES users (id) ON DELETE SET NULL
86 );
87 CREATE INDEX ix_exams_status ON exams (status);
88
89 -----
90 -- login_access_requests
91 -----
92 CREATE TABLE login_access_requests (
93 candidate_id UUID NOT NULL,
94 status login_access_status NOT NULL,
95 requested_at TIMESTAMP WITH TIME ZONE NOT NULL,
96 reviewed_at TIMESTAMP WITH TIME ZONE,
97 reviewed_by_id UUID,
98 review_note TEXT,
99 id UUID NOT NULL,
100 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
101 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
102 CONSTRAINT pk_login_access_requests PRIMARY KEY (id),
103 CONSTRAINT fk_login_access_requests_candidate_id_users FOREIGN KEY(candidate_id) REFERENCES users (id) ON DELETE
CASCADE,
104 CONSTRAINT fk_login_access_requests_reviewed_by_id_users FOREIGN KEY(reviewed_by_id) REFERENCES users (id) ON DELETE
SET NULL
105 );
106 CREATE UNIQUE INDEX ix_login_access_requests_candidate_id ON login_access_requests (candidate_id);
107 CREATE INDEX ix_login_access_requests_status ON login_access_requests (status);
108
109 -----
110 -- questions
111 -----
112 CREATE TABLE questions (
113 subject_id UUID NOT NULL,
114 question_type question_type NOT NULL,
115 difficulty difficulty NOT NULL,
116 body TEXT NOT NULL,
117 model_answer TEXT,
118 rubric JSONB,
119 min_words INTEGER,
120 max_words INTEGER,
121 marks FLOAT NOT NULL,
122 negative_marks FLOAT NOT NULL,
123 tags JSONB,
124 is_active BOOLEAN NOT NULL,
125 created_by_id UUID,
126 id UUID NOT NULL,
127 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
128 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
129 CONSTRAINT pk_questions PRIMARY KEY (id),
130 CONSTRAINT fk_questions_subject_id_subjects FOREIGN KEY(subject_id) REFERENCES subjects (id) ON DELETE RESTRICT,
131 CONSTRAINT fk_questions_created_by_id_users FOREIGN KEY(created_by_id) REFERENCES users (id) ON DELETE SET NULL
132 );
133 CREATE INDEX ix_questions_is_active ON questions (is_active);
134 CREATE INDEX ix_questions_pool_lookup ON questions (subject_id, question_type, difficulty);
135
136 -----
137 -- exam_enrollments
138 -----
139 CREATE TABLE exam_enrollments (
140 exam_id UUID NOT NULL,
141 candidate_id UUID NOT NULL,

```

```

142 assigned_by_id UUID,
143 assigned_at TIMESTAMP WITH TIME ZONE NOT NULL,
144 id UUID NOT NULL,
145 CONSTRAINT pk_exam_enrollments PRIMARY KEY (id),
146 CONSTRAINT uq_exam_enrollment UNIQUE (exam_id, candidate_id),
147 CONSTRAINT fk_exam_enrollments_exam_id_exams FOREIGN KEY(exam_id) REFERENCES exams (id) ON DELETE CASCADE,
148 CONSTRAINT fk_exam_enrollments_candidate_id_users FOREIGN KEY(candidate_id) REFERENCES users (id) ON DELETE CASCADE,
149 CONSTRAINT fk_exam_enrollments_assigned_by_id_users FOREIGN KEY(assigned_by_id) REFERENCES users (id) ON DELETE SET
NULL
150 );
151 CREATE INDEX ix_exam_enrollments_candidate_id ON exam_enrollments (candidate_id);
152 CREATE INDEX ix_exam_enrollments_exam_id ON exam_enrollments (exam_id);
153
154 -- -----
155 -- exam_questions
156 -- -----
157 CREATE TABLE exam_questions (
158 exam_id UUID NOT NULL,
159 question_id UUID NOT NULL,
160 marks_override FLOAT,
161 order_index INTEGER NOT NULL,
162 id UUID NOT NULL,
163 CONSTRAINT pk_exam_questions PRIMARY KEY (id),
164 CONSTRAINT uq_exam_question UNIQUE (exam_id, question_id),
165 CONSTRAINT fk_exam_questions_exam_id_exams FOREIGN KEY(exam_id) REFERENCES exams (id) ON DELETE CASCADE,
166 CONSTRAINT fk_exam_questions_question_id_questions FOREIGN KEY(question_id) REFERENCES questions (id) ON DELETE
RESTRICT
167 );
168 CREATE INDEX ix_exam_questions_exam_id ON exam_questions (exam_id);
169
170 -- -----
171 -- exam_sessions
172 -- -----
173 CREATE TABLE exam_sessions (
174 exam_id UUID NOT NULL,
175 candidate_id UUID NOT NULL,
176 paper_seed VARCHAR(64) NOT NULL,
177 question_order JSONB NOT NULL,
178 started_at TIMESTAMP WITH TIME ZONE NOT NULL,
179 expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
180 submitted_at TIMESTAMP WITH TIME ZONE,
181 last_heartbeat_at TIMESTAMP WITH TIME ZONE,
182 status session_status NOT NULL,
183 suspicion_score FLOAT NOT NULL,
184 tab_switch_count INTEGER NOT NULL,
185 is_flagged BOOLEAN NOT NULL,
186 termination_reason TEXT,
187 id UUID NOT NULL,
188 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
189 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
190 CONSTRAINT pk_exam_sessions PRIMARY KEY (id),
191 CONSTRAINT uq_exam_session_attempt UNIQUE (exam_id, candidate_id),
192 CONSTRAINT fk_exam_sessions_exam_id_exams FOREIGN KEY(exam_id) REFERENCES exams (id) ON DELETE CASCADE,
193 CONSTRAINT fk_exam_sessions_candidate_id_users FOREIGN KEY(candidate_id) REFERENCES users (id) ON DELETE CASCADE
194 );
195 CREATE INDEX ix_exam_sessions_candidate_id ON exam_sessions (candidate_id);
196 CREATE INDEX ix_exam_sessions_exam_id ON exam_sessions (exam_id);
197 CREATE INDEX ix_exam_sessions_is_flagged ON exam_sessions (is_flagged);
198 CREATE INDEX ix_exam_sessions_status ON exam_sessions (status);
199
200 -- -----
201 -- question_options
202 -- -----
203 CREATE TABLE question_options (
204 question_id UUID NOT NULL,
205 text TEXT NOT NULL,
206 is_correct BOOLEAN NOT NULL,
207 order_index INTEGER NOT NULL,
208 id UUID NOT NULL,
209 CONSTRAINT pk_question_options PRIMARY KEY (id),
210 CONSTRAINT fk_question_options_question_id_questions FOREIGN KEY(question_id) REFERENCES questions (id) ON DELETE
CASCADE
211 );
212
213 -- -----
214 -- answers
215 -- -----

```

```

216 CREATE TABLE answers (
217     session_id UUID NOT NULL,
218     question_id UUID NOT NULL,
219     selected_option_ids JSONB,
220     text_answer TEXT,
221     image_object_key VARCHAR(512),
222     image_thumb_key VARCHAR(512),
223     ocr_text TEXT,
224     ocr_confidence FLOAT,
225     word_count INTEGER,
226     awarded_marks FLOAT,
227     max_marks FLOAT NOT NULL,
228     grade_status grade_status NOT NULL,
229     examiner_comment TEXT,
230     graded_by_id UUID,
231     graded_at TIMESTAMP WITH TIME ZONE,
232     answered_at TIMESTAMP WITH TIME ZONE,
233     id UUID NOT NULL,
234     created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
235     updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
236     CONSTRAINT pk_answers PRIMARY KEY (id),
237     CONSTRAINT uq_answer_per_question UNIQUE (session_id, question_id),
238     CONSTRAINT fk_answers_session_id_exam_sessions FOREIGN KEY(session_id) REFERENCES exam_sessions (id) ON DELETE
CASCADE,
239     CONSTRAINT fk_answers_question_id_questions FOREIGN KEY(question_id) REFERENCES questions (id) ON DELETE RESTRICT,
240     CONSTRAINT fk_answers_graded_by_id_users FOREIGN KEY(graded_by_id) REFERENCES users (id) ON DELETE SET NULL
241 );
242 CREATE INDEX ix_answers_grade_status ON answers (grade_status);
243 CREATE INDEX ix_answers_session_id ON answers (session_id);
244
245 -- -----
246 -- proctor_events
247 -- -----
248 CREATE TABLE proctor_events (
249     session_id UUID NOT NULL,
250     event_type proctor_event_type NOT NULL,
251     severity proctor_severity NOT NULL,
252     occurred_at TIMESTAMP WITH TIME ZONE NOT NULL,
253     server_received_at TIMESTAMP WITH TIME ZONE NOT NULL,
254     duration_ms INTEGER,
255     weight FLOAT NOT NULL,
256     metadata JSONB,
257     snapshot_object_key VARCHAR(512),
258     id UUID NOT NULL,
259     CONSTRAINT pk_proctor_events PRIMARY KEY (id),
260     CONSTRAINT fk_proctor_events_session_id_exam_sessions FOREIGN KEY(session_id) REFERENCES exam_sessions (id) ON DELETE
CASCADE
261 );
262 CREATE INDEX ix_proctor_events_timeline ON proctor_events (session_id, occurred_at);
263
264 -- -----
265 -- results
266 -- -----
267 CREATE TABLE results (
268     session_id UUID NOT NULL,
269     total_marks FLOAT NOT NULL,
270     obtained_marks FLOAT NOT NULL,
271     percentage FLOAT NOT NULL,
272     correct_count INTEGER NOT NULL,
273     incorrect_count INTEGER NOT NULL,
274     unanswered_count INTEGER NOT NULL,
275     pending_review_count INTEGER NOT NULL,
276     published BOOLEAN NOT NULL,
277     published_at TIMESTAMP WITH TIME ZONE,
278     id UUID NOT NULL,
279     created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
280     updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
281     CONSTRAINT pk_results PRIMARY KEY (id),
282     CONSTRAINT uq_results_session_id UNIQUE (session_id),
283     CONSTRAINT fk_results_session_id_exam_sessions FOREIGN KEY(session_id) REFERENCES exam_sessions (id) ON DELETE CASCADE
284 );
285 CREATE INDEX ix_results_published ON results (published);
286
287 -- -----
288 -- ai_evaluations
289 -- -----
290 CREATE TABLE ai_evaluations (

```

```
291 answer_id UUID NOT NULL,
292 provider VARCHAR(50) NOT NULL,
293 model VARCHAR(100),
294 score FLOAT NOT NULL,
295 max_score FLOAT NOT NULL,
296 justification TEXT NOT NULL,
297 confidence FLOAT NOT NULL,
298 raw_response JSONB,
299 error TEXT,
300 id UUID NOT NULL,
301 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
302 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
303 CONSTRAINT pk_ai_evaluations PRIMARY KEY (id),
304 CONSTRAINT fk_ai_evaluations_answer_id_answers FOREIGN KEY(answer_id) REFERENCES answers (id) ON DELETE CASCADE
305 );
306 CREATE INDEX ix_ai_evaluations_answer_id ON ai_evaluations (answer_id);
```

II Declarative model sources

The SQLAlchemy 2.0 declarative definitions, verbatim. These are the authority: the DDL in Part I is derived from them, and Alembic diffs against them.

base.py – declarative base, naming convention, mixins

backend/app/db/base.py · 41 lines

```
1  """Declarative base, naming conventions and shared mixins."""
2
3  from __future__ import annotations
4
5  import uuid
6  from datetime import datetime
7
8  from sqlalchemy import DateTime, MetaData, func
9  from sqlalchemy.dialects.postgresql import UUID as PGUUID
10 from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
11
12 # Explicit constraint naming so Alembic autogenerate produces stable, revertible names.
13 NAMING_CONVENTION = {
14     "ix": "ix_(column_0_label)s",
15     "uq": "uq_(table_name)s_(column_0_name)s",
16     "ck": "ck_(table_name)s_(constraint_name)s",
17     "fk": "fk_(table_name)s_(column_0_name)s_(referred_table_name)s",
18     "pk": "pk_(table_name)s",
19 }
20
21
22 class Base(DeclarativeBase):
23     metadata = MetaData(naming_convention=NAMING_CONVENTION)
24
25
26 class UUIDPrimaryKeyMixin:
27     id: Mapped[uuid.UUID] = mapped_column(
28         PGUUID(as_uuid=True), primary_key=True, default=uuid.uuid4
29     )
30
31
32 class TimestampMixin:
33     created_at: Mapped[datetime] = mapped_column(
34         DateTime(timezone=True), server_default=func.now(), nullable=False
35     )
36     updated_at: Mapped[datetime] = mapped_column(
37         DateTime(timezone=True),
38         server_default=func.now(),
39         onupdate=func.now(),
40         nullable=False,
41     )
```

session.py – engine and session factory

backend/app/db/session.py · 34 lines

```
1  """Engine and session factory (sync SQLAlchemy 2.0 + psycopg 3)."""
2
3  from __future__ import annotations
4
5  from collections.abc import Iterator
6
7  from sqlalchemy import create_engine
8  from sqlalchemy.orm import Session, sessionmaker
9
10 from app.core.config import settings
11
12 engine = create_engine(
13     settings.DATABASE_URL,
14     pool_size=settings.DB_POOL_SIZE,
15     max_overflow=settings.DB_MAX_OVERFLOW,
16     pool_pre_ping=True,
17     echo=settings.SQL_ECHO,
18     future=True,
19 )
20
21 SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, future=True)
22
23
24 def get_db() -> Iterator[Session]:
25     """FastAPI dependency: one session per request, rolled back on error."""
26     db = SessionLocal()
27     try:
28         yield db
29         db.commit()
30     except Exception:
31         db.rollback()
32         raise
33     finally:
34         db.close()
```


models/__init__.py – registry

backend/app/db/models/__init__.py · 57 lines

```
1  """Model package. Importing this registers every table on ``Base.metadata``."""
2
3  from app.db.base import Base
4  from app.db.models.enrollment import ExamEnrollment
5  from app.db.models.enums import (
6      AccessStatus,
7      Difficulty,
8      ExamStatus,
9      GradeStatus,
10     LoginAccessStatus,
11     ProctorEventType,
12     ProctorSeverity,
13     QuestionType,
14     SessionStatus,
15     UserRole,
16 )
17 from app.db.models.exam import (
18     DEFAULT_GRADING_CONFIG,
19     DEFAULT_PROCTOR_CONFIG,
20     Exam,
21     ExamQuestion,
22 )
23 from app.db.models.exam_session import Answer, ExamSession
24 from app.db.models.grading import AiEvaluation, Result
25 from app.db.models.login_access import LoginAccessRequest
26 from app.db.models.proctor import ProctorEvent
27 from app.db.models.question import Question, QuestionOption, Subject
28 from app.db.models.user import User
29
30 __all__ = [
31     "DEFAULT_GRADING_CONFIG",
32     "DEFAULT_PROCTOR_CONFIG",
33     "AccessStatus",
34     "AiEvaluation",
35     "Answer",
36     "Base",
37     "Difficulty",
38     "Exam",
39     "ExamQuestion",
40     "ExamEnrollment",
41     "ExamSession",
42     "ExamStatus",
43     "GradeStatus",
44     "LoginAccessRequest",
45     "LoginAccessStatus",
46     "ProctorEvent",
47     "ProctorEventType",
48     "ProctorSeverity",
49     "Question",
50     "QuestionOption",
51     "QuestionType",
52     "Result",
53     "SessionStatus",
54     "Subject",
55     "User",
56     "UserRole",
57 ]
```

models/enums.py – domain enums

backend/app/db/models/enums.py · 99 lines

```
1  """Domain enums. These map to native PostgreSQL enum types."""
2
3  from __future__ import annotations
4
5  import enum
6
7
8  class UserRole(str, enum.Enum):
9      ADMIN = "admin"
10     EXAMINER = "examiner"
11     CANDIDATE = "candidate"
12
13
14  class AccessStatus(str, enum.Enum):
15      """Admin-controlled gate.
16
17      Examiners register as ``PENDING`` and cannot touch the question bank or any exam
18      until an admin flips them to ``APPROVED``. Candidates are approved on registration.
19      """
20
21     PENDING = "pending"
22     APPROVED = "approved"
23     REVOKED = "revoked"
24
25
26  class LoginAccessStatus(str, enum.Enum):
27      """Permission to *use* the platform – deliberately separate from account state.
28
29      A candidate's account is created active and stays active; this is the gate that an
30      administrator (or an authorised examiner) opens. Revoking access sets an approved
31      request back to ``REJECTED`` with a review note, so there is exactly one axis to
32      reason about when deciding whether someone may log in.
33      """
34
35     PENDING = "pending"
36     APPROVED = "approved"
37     REJECTED = "rejected"
38
39
40  class QuestionType(str, enum.Enum):
41     MCQ = "mcq"
42     MULTI_SELECT = "multi_select"
43     SHORT_ANSWER = "short_answer"
44     LONG_ANSWER = "long_answer"
45     IMAGE_UPLOAD = "image_upload"
46
47
48  OBJECTIVE_TYPES = {QuestionType.MCQ, QuestionType.MULTI_SELECT}
49  SUBJECTIVE_TYPES = {
50     QuestionType.SHORT_ANSWER,
51     QuestionType.LONG_ANSWER,
52     QuestionType.IMAGE_UPLOAD,
53 }
54
55
56  class Difficulty(str, enum.Enum):
57     EASY = "easy"
58     MEDIUM = "medium"
59     HARD = "hard"
60
61
62  class ExamStatus(str, enum.Enum):
63     DRAFT = "draft"
64     PUBLISHED = "published"
65     CLOSED = "closed"
66
67
68  class SessionStatus(str, enum.Enum):
69     IN_PROGRESS = "in_progress"
70     SUBMITTED = "submitted"
71     AUTO_SUBMITTED = "auto_submitted"
72     TERMINATED = "terminated"
```

```
73
74
75 class GradeStatus(str, enum.Enum):
76     UNANSWERED = "unanswered"
77     AUTO_SCORED = "auto_scored"
78     PENDING_AI = "pending_ai"
79     AI_SCORED = "ai_scored"
80     EXAMINER_REVIEWED = "examiner_reviewed"
81
82
83 class ProctorEventType(str, enum.Enum):
84     FACE_MISSING = "face_missing"
85     MULTIPLE_FACES = "multiple_faces"
86     GAZE_AWAY = "gaze_away"
87     TAB_SWITCH = "tab_switch"
88     WINDOW_BLUR = "window_blur"
89     FULLSCREEN_EXIT = "fullscreen_exit"
90     CAMERA_BLOCKED = "camera_blocked"
91     PASTE_ATTEMPT = "paste_attempt"
92     COPY_ATTEMPT = "copy_attempt"
93     DEVTOOLS_OPEN = "devtools_open"
94
95
96 class ProctorSeverity(str, enum.Enum):
97     INFO = "info"
98     WARNING = "warning"
99     CRITICAL = "critical"
```

models/user.py – User

backend/app/db/models/user.py · 95 lines

```
1  """Users and the admin-controlled access gate."""
2
3  from __future__ import annotations
4
5  import uuid
6  from datetime import datetime
7  from typing import TYPE_CHECKING
8
9  from sqlalchemy import Boolean, DateTime, Enum, ForeignKey, String, Text
10 from sqlalchemy.dialects.postgresql import UUID as PGUUID
11 from sqlalchemy.orm import Mapped, mapped_column, relationship
12
13 from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
14 from app.db.models.enums import AccessStatus, UserRole
15
16 if TYPE_CHECKING:
17     from app.db.models.enrollment import ExamEnrollment
18     from app.db.models.exam import Exam
19     from app.db.models.exam_session import ExamSession
20     from app.db.models.login_access import LoginAccessRequest
21
22
23 class User(UUIDPrimaryKeyMixin, TimestampMixin, Base):
24     __tablename__ = "users"
25
26     email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
27     password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
28     # first/last are the source of truth for greetings and sorting; full_name is kept
29     # in sync so existing queries, exports and UI that read it keep working.
30     first_name: Mapped[str] = mapped_column(String(80), nullable=False, default="")
31     last_name: Mapped[str] = mapped_column(String(80), nullable=False, default="")
32     full_name: Mapped[str] = mapped_column(String(150), nullable=False)
33     role: Mapped[UserRole] = mapped_column(
34         Enum(UserRole, name="user_role", values_callable=lambda e: [m.value for m in e]),
35         nullable=False,
36         index=True,
37     )
38     is_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
39
40     # --- Admin access gate -----
41     # An examiner is inert until an admin approves them. Candidates are approved
42     # at registration; admins are approved by definition.
43     access_status: Mapped[AccessStatus] = mapped_column(
44         Enum(AccessStatus, name="access_status", values_callable=lambda e: [m.value for m in e]),
45         default=AccessStatus.PENDING,
46         nullable=False,
47         index=True,
48     )
49     access_changed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
50     access_changed_by_id: Mapped[uuid.UUID | None] = mapped_column(
51         PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
52     )
53     access_note: Mapped[str | None] = mapped_column(Text)
54
55     last_login_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
56
57     access_changed_by: Mapped[User | None] = relationship(
58         remote_side="User.id", foreign_keys=[access_changed_by_id]
59     )
60     exam_sessions: Mapped[list[ExamSession]] = relationship(
61         back_populates="candidate",
62         foreign_keys="ExamSession.candidate_id",
63         cascade="all, delete-orphan",
64     )
65     exams_created: Mapped[list[Exam]] = relationship(
66         back_populates="created_by", foreign_keys="Exam.created_by_id"
67     )
68     login_access_request: Mapped[LoginAccessRequest | None] = relationship(
69         back_populates="candidate",
70         foreign_keys="LoginAccessRequest.candidate_id",
71         cascade="all, delete-orphan",
72         uselist=False,
```

```

73     )
74     enrollments: Mapped[list[ExamEnrollment]] = relationship(
75         back_populates="candidate",
76         foreign_keys="ExamEnrollment.candidate_id",
77         cascade="all, delete-orphan",
78     )
79
80     @property
81     def is_approved(self) -> bool:
82         return self.access_status is AccessStatus.APPROVED
83
84     @property
85     def display_first_name(self) -> str:
86         """First name for greetings, falling back to the first token of full_name."""
87         return self.first_name or self.full_name.split(" ")[0]
88
89     def set_name(self, first: str, last: str) -> None:
90         self.first_name = first.strip()
91         self.last_name = last.strip()
92         self.full_name = " ".join(part for part in (self.first_name, self.last_name) if part)
93
94     def __repr__(self) -> str: # pragma: no cover - debugging aid
95         return f"<User {self.email} role={self.role.value} access={self.access_status.value}>"

```

models/login_access.py – LoginAccessRequest

backend/app/db/models/login_access.py · 67 lines

```
1  """Login approval – permission to use the platform, distinct from account state.
2
3  A candidate's account is created **active** and stays active. What is gated is the *login*:
4  on their first sign-in attempt with valid credentials, a request is raised here and the
5  candidate is held on a pending screen until an administrator – or an examiner authorised
6  over them – decides.
7
8  Once approved, it stays approved: later logins reuse the same row and never queue again
9  unless someone explicitly revokes it (which moves the row back to REJECTED with a note).
10 """
11
12 from __future__ import annotations
13
14 import uuid
15 from datetime import datetime
16 from typing import TYPE_CHECKING
17
18 from sqlalchemy import DateTime, Enum, ForeignKey, Text
19 from sqlalchemy.dialects.postgresql import UUID as PGUUID
20 from sqlalchemy.orm import Mapped, mapped_column, relationship
21
22 from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
23 from app.db.models.enums import LoginAccessStatus
24
25 if TYPE_CHECKING:
26     from app.db.models.user import User
27
28
29 class LoginAccessRequest(UUIDPrimaryKeyMixin, TimestampMixin, Base):
30     __tablename__ = "login_access_requests"
31
32     # One standing request per candidate: approval is a durable state, not an event log.
33     candidate_id: Mapped[uuid.UUID] = mapped_column(
34         PGUUID(as_uuid=True),
35         ForeignKey("users.id", ondelete="CASCADE"),
36         nullable=False,
37         unique=True,
38         index=True,
39     )
40     status: Mapped[LoginAccessStatus] = mapped_column(
41         Enum(
42             LoginAccessStatus,
43             name="login_access_status",
44             values_callable=lambda e: [m.value for m in e],
45         ),
46         default=LoginAccessStatus.PENDING,
47         nullable=False,
48         index=True,
49     )
50     requested_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
51     reviewed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
52     reviewed_by_id: Mapped[uuid.UUID | None] = mapped_column(
53         PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
54     )
55     review_note: Mapped[str | None] = mapped_column(Text)
56
57     candidate: Mapped[User] = relationship(
58         back_populates="login_access_request", foreign_keys=[candidate_id]
59     )
60     reviewed_by: Mapped[User | None] = relationship(foreign_keys=[reviewed_by_id])
61
62     @property
63     def is_approved(self) -> bool:
64         return self.status is LoginAccessStatus.APPROVED
65
66     def __repr__(self) -> str: # pragma: no cover – debugging aid
67         return f"<LoginAccessRequest {self.candidate_id} {self.status.value}>"
```

models/enrollment.py – ExamEnrollment

backend/app/db/models/enrollment.py · 52 lines

```
1  """Exam enrolment – which candidates a given exam is actually assigned to.
2
3  Approved login is permission to use the platform; it is *not* permission to sit every
4  paper on it. A candidate sees and can start only the exams they are enrolled in, and this
5  table is also what defines an examiner's authority over a candidate: an examiner may
6  review a candidate's login request only if that candidate is enrolled in one of the
7  examiner's own exams.
8  """
9
10 from __future__ import annotations
11
12 import uuid
13 from datetime import datetime
14 from typing import TYPE_CHECKING
15
16 from sqlalchemy import DateTime, ForeignKey, UniqueConstraint
17 from sqlalchemy.dialects.postgresql import UUID as PGUUID
18 from sqlalchemy.orm import Mapped, mapped_column, relationship
19
20 from app.db.base import Base, UUIDPrimaryKeyMixin
21
22 if TYPE_CHECKING:
23     from app.db.models.exam import Exam
24     from app.db.models.user import User
25
26
27 class ExamEnrollment(UUIDPrimaryKeyMixin, Base):
28     __tablename__ = "exam_enrollments"
29     __table_args__ = (UniqueConstraint("exam_id", "candidate_id", name="uq_exam_enrollment"),)
30
31     exam_id: Mapped[uuid.UUID] = mapped_column(
32         PGUUID(as_uuid=True),
33         ForeignKey("exams.id", ondelete="CASCADE"),
34         nullable=False,
35         index=True,
36     )
37     candidate_id: Mapped[uuid.UUID] = mapped_column(
38         PGUUID(as_uuid=True),
39         ForeignKey("users.id", ondelete="CASCADE"),
40         nullable=False,
41         index=True,
42     )
43     assigned_by_id: Mapped[uuid.UUID | None] = mapped_column(
44         PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
45     )
46     assigned_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
47
48     exam: Mapped[Exam] = relationship(back_populates="enrollments")
49     candidate: Mapped[User] = relationship(
50         back_populates="enrollments", foreign_keys=[candidate_id]
51     )
52     assigned_by: Mapped[User | None] = relationship(foreign_keys=[assigned_by_id])
```

models/question.py – Subject, Question, QuestionOption

backend/app/db/models/question.py · 88 lines

```
1  """Subjects, question bank and options."""
2
3  from __future__ import annotations
4
5  import uuid
6  from typing import TYPE_CHECKING, Any
7
8  from sqlalchemy import Boolean, Enum, Float, ForeignKey, Index, Integer, String, Text
9  from sqlalchemy.dialects.postgresql import JSONB
10 from sqlalchemy.dialects.postgresql import UUID as PGUUID
11 from sqlalchemy.orm import Mapped, mapped_column, relationship
12
13 from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
14 from app.db.models.enums import Difficulty, QuestionType
15
16 if TYPE_CHECKING:
17     from app.db.models.user import User
18
19
20 class Subject(UUIDPrimaryKeyMixin, TimestampMixin, Base):
21     __tablename__ = "subjects"
22
23     code: Mapped[str] = mapped_column(String(32), unique=True, index=True, nullable=False)
24     name: Mapped[str] = mapped_column(String(150), nullable=False)
25     description: Mapped[str | None] = mapped_column(Text)
26
27     questions: Mapped[list[Question]] = relationship(back_populates="subject")
28
29
30 class Question(UUIDPrimaryKeyMixin, TimestampMixin, Base):
31     __tablename__ = "questions"
32     __table_args__ = (
33         Index("ix_questions_pool_lookup", "subject_id", "question_type", "difficulty"),
34     )
35
36     subject_id: Mapped[uuid.UUID] = mapped_column(
37         PGUUID(as_uuid=True), ForeignKey("subjects.id", ondelete="RESTRICT"), nullable=False
38     )
39     question_type: Mapped[QuestionType] = mapped_column(
40         Enum(QuestionType, name="question_type", values_callable=lambda e: [m.value for m in e]),
41         nullable=False,
42     )
43     difficulty: Mapped[Difficulty] = mapped_column(
44         Enum(Difficulty, name="difficulty", values_callable=lambda e: [m.value for m in e]),
45         nullable=False,
46         default=Difficulty.MEDIUM,
47     )
48     body: Mapped[str] = mapped_column(Text, nullable=False)
49     # Reference answer for subjective types; also shown to the examiner while grading.
50     model_answer: Mapped[str | None] = mapped_column(Text)
51     # Free-form rubric handed to the grader, e.g. {"criteria": [{"name": "...", "weight": ...}]}
52     rubric: Mapped[dict[str, Any] | None] = mapped_column(JSONB)
53     # Word bounds for written answers. Enforced server-side on every autosave, and shown
54     # as a live counter in the runner so a candidate is never surprised at submit time.
55     min_words: Mapped[int | None] = mapped_column(Integer)
56     max_words: Mapped[int | None] = mapped_column(Integer)
57     marks: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
58     negative_marks: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
59     tags: Mapped[list[str] | None] = mapped_column(JSONB)
60     is_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False, index=True)
61
62     created_by_id: Mapped[uuid.UUID | None] = mapped_column(
63         PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
64     )
65
66     subject: Mapped[Subject] = relationship(back_populates="questions")
67     created_by: Mapped[User | None] = relationship()
68     options: Mapped[list[QuestionOption]] = relationship(
69         back_populates="question",
70         cascade="all, delete-orphan",
71         order_by="QuestionOption.order_index",
72     )
```



```
73
74     def __repr__(self) -> str: # pragma: no cover - debugging aid
75         return f"<Question {self.question_type.value} {self.body[:40]!r}>"
76
77
78 class QuestionOption(UUIDPrimaryKeyMixin, Base):
79     __tablename__ = "question_options"
80
81     question_id: Mapped[uuid.UUID] = mapped_column(
82         PGUUID(as_uuid=True), ForeignKey("questions.id", ondelete="CASCADE"), nullable=False
83     )
84     text: Mapped[str] = mapped_column(Text, nullable=False)
85     is_correct: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
86     order_index: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
87
88     question: Mapped[Question] = relationship(back_populates="options")
```

models/exam.py – Exam, ExamQuestion

backend/app/db/models/exam.py · 145 lines

```
1  """Exams, their configuration, and the question pool bound to each exam."""
2
3  from __future__ import annotations
4
5  import uuid
6  from datetime import datetime
7  from typing import TYPE_CHECKING, Any
8
9  from sqlalchemy import (
10     Boolean,
11     DateTime,
12     Enum,
13     Float,
14     ForeignKey,
15     Integer,
16     String,
17     Text,
18     UniqueConstraint,
19 )
20 from sqlalchemy.dialects.postgresql import JSONB
21 from sqlalchemy.dialects.postgresql import UUID as PGUUID
22 from sqlalchemy.orm import Mapped, mapped_column, relationship
23
24 from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
25 from app.db.models.enums import ExamStatus
26
27 if TYPE_CHECKING:
28     from app.db.models.enrollment import ExamEnrollment
29     from app.db.models.exam_session import ExamSession
30     from app.db.models.question import Question, Subject
31     from app.db.models.user import User
32
33
34 DEFAULT_PROCTOR_CONFIG: dict[str, Any] = {
35     "webcam_enabled": True,
36     "gaze_tracking_enabled": True,
37     "gaze_sensitivity": 0.6, # 0..1, higher = stricter
38     "max_tab_switches": 3, # warnings before the session is auto-flagged
39     "terminate_on_score": 100.0, # suspicion score that ends the session
40     "flag_on_score": 45.0, # suspicion score that flags for review
41     "snapshot_interval_seconds": 60,
42     "require_fullscreen": True,
43     "block_copy_paste": True,
44     "weights": {
45         "face_missing": 6.0,
46         "multiple_faces": 15.0,
47         "gaze_away": 3.0,
48         "tab_switch": 10.0,
49         "window_blur": 4.0,
50         "fullscreen_exit": 8.0,
51         "camera_blocked": 20.0,
52         "paste_attempt": 12.0,
53         "copy_attempt": 6.0,
54         "devtools_open": 25.0,
55     },
56 }
57
58 DEFAULT_GRADING_CONFIG: dict[str, Any] = {
59     "partial_credit_multi_select": False,
60     "auto_publish_results": False,
61     "grader_provider": None, # None -> fall back to the app-level setting
62     "grader_model": None,
63 }
64
65
66 class Exam(UUIDPrimaryKeyMixin, TimestampMixin, Base):
67     __tablename__ = "exams"
68
69     subject_id: Mapped[uuid.UUID] = mapped_column(
70         PGUUID(as_uuid=True), ForeignKey("subjects.id", ondelete="RESTRICT"), nullable=False
71     )
72     title: Mapped[str] = mapped_column(String(200), nullable=False)
```

```

73     description: Mapped[str | None] = mapped_column(Text)
74
75     duration_minutes: Mapped[int] = mapped_column(Integer, nullable=False)
76     starts_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
77     ends_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
78
79     # {"rules": [{"question_type": "mcq", "difficulty": "easy", "count": 10}, ...]}
80     selection_rules: Mapped[dict[str, Any]] = mapped_column(JSONB, nullable=False, default=dict)
81     randomize: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
82     shuffle_options: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
83     negative_marking: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
84     # Stable per-exam salt so the deterministic paper seed cannot be guessed from ids alone.
85     paper_salt: Mapped[str] = mapped_column(String(64), nullable=False)
86
87     proctor_config: Mapped[dict[str, Any]] = mapped_column(
88         JSONB, nullable=False, default=lambda: dict(DEFAULT_PROCTOR_CONFIG)
89     )
90     grading_config: Mapped[dict[str, Any]] = mapped_column(
91         JSONB, nullable=False, default=lambda: dict(DEFAULT_GRADING_CONFIG)
92     )
93
94     status: Mapped[ExamStatus] = mapped_column(
95         Enum(ExamStatus, name="exam_status", values_callable=lambda e: [m.value for m in e]),
96         default=ExamStatus.DRAFT,
97         nullable=False,
98         index=True,
99     )
100     results_published: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
101
102     created_by_id: Mapped[uuid.UUID | None] = mapped_column(
103         PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
104     )
105
106     subject: Mapped[Subject] = relationship()
107     created_by: Mapped[User | None] = relationship(
108         back_populates="exams_created", foreign_keys=[created_by_id]
109     )
110     exam_questions: Mapped[list[ExamQuestion]] = relationship(
111         back_populates="exam", cascade="all, delete-orphan"
112     )
113     sessions: Mapped[list[ExamSession]] = relationship(
114         back_populates="exam", cascade="all, delete-orphan"
115     )
116     enrollments: Mapped[list[ExamEnrollment]] = relationship(
117         back_populates="exam", cascade="all, delete-orphan"
118     )
119
120     @property
121     def total_marks(self) -> float:
122         return sum(eq.effective_marks for eq in self.exam_questions)
123
124
125 class ExamQuestion(UUIDPrimaryKeyMixin, Base):
126     """The pool an exam draws from. A candidate's paper is a subset of these."""
127
128     __tablename__ = "exam_questions"
129     __table_args__ = (UniqueConstraint("exam_id", "question_id", name="uq_exam_question"),)
130
131     exam_id: Mapped[uuid.UUID] = mapped_column(
132         PGUUID(as_uuid=True), ForeignKey("exams.id", ondelete="CASCADE"), nullable=False, index=True
133     )
134     question_id: Mapped[uuid.UUID] = mapped_column(
135         PGUUID(as_uuid=True), ForeignKey("questions.id", ondelete="RESTRICT"), nullable=False
136     )
137     marks_override: Mapped[float | None] = mapped_column(Float)
138     order_index: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
139
140     exam: Mapped[Exam] = relationship(back_populates="exam_questions")
141     question: Mapped[Question] = relationship()
142
143     @property
144     def effective_marks(self) -> float:
145         return self.marks_override if self.marks_override is not None else self.question.marks

```

models/exam_session.py – ExamSession, Answer

backend/app/db/models/exam_session.py · 150 lines

```
1  """Live exam sessions and the answers recorded inside them."""
2
3  from __future__ import annotations
4
5  import uuid
6  from datetime import datetime
7  from typing import TYPE_CHECKING, Any
8
9  from sqlalchemy import (
10     DateTime,
11     Enum,
12     Float,
13     ForeignKey,
14     Integer,
15     String,
16     Text,
17     UniqueConstraint,
18 )
19 from sqlalchemy.dialects.postgresql import JSONB
20 from sqlalchemy.dialects.postgresql import UUID as PGUUID
21 from sqlalchemy.orm import Mapped, mapped_column, relationship
22
23 from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
24 from app.db.models.enums import GradeStatus, SessionStatus
25
26 if TYPE_CHECKING:
27     from app.db.models.exam import Exam
28     from app.db.models.grading import AiEvaluation, Result
29     from app.db.models.proctor import ProctorEvent
30     from app.db.models.question import Question
31     from app.db.models.user import User
32
33
34 class ExamSession(UUIDPrimaryKeyMixin, TimestampMixin, Base):
35     """One candidate's attempt at one exam.
36
37     The unique (exam_id, candidate_id) constraint is what enforces a single attempt.
38     ``question_order`` is persisted at start so a server restart mid-exam cannot reshuffle
39     the paper under the candidate.
40     """
41
42     __tablename__ = "exam_sessions"
43     __table_args__ = (UniqueConstraint("exam_id", "candidate_id", name="uq_exam_session_attempt"),)
44
45     exam_id: Mapped[uuid.UUID] = mapped_column(
46         PGUUID(as_uuid=True), ForeignKey("exams.id", ondelete="CASCADE"), nullable=False, index=True
47     )
48     candidate_id: Mapped[uuid.UUID] = mapped_column(
49         PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="CASCADE"), nullable=False, index=True
50     )
51
52     paper_seed: Mapped[str] = mapped_column(String(64), nullable=False)
53     # [{"question_id": "...", "option_order": [{"...", "..."}], ...}]
54     question_order: Mapped[list[dict[str, Any]]] = mapped_column(JSONB, nullable=False)
55
56     started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
57     expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
58     submitted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
59     last_heartbeat_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
60
61     status: Mapped[SessionStatus] = mapped_column(
62         Enum(SessionStatus, name="session_status", values_callable=lambda e: [m.value for m in e]),
63         default=SessionStatus.IN_PROGRESS,
64         nullable=False,
65         index=True,
66     )
67
68     suspicion_score: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
69     tab_switch_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
70     is_flagged: Mapped[bool] = mapped_column(default=False, nullable=False, index=True)
71     termination_reason: Mapped[str | None] = mapped_column(Text)
72
```

```

73 exam: Mapped[Exam] = relationship(back_populates="sessions")
74 candidate: Mapped[User] = relationship(
75     back_populates="exam_sessions", foreign_keys=[candidate_id]
76 )
77 answers: Mapped[list[Answer]] = relationship(
78     back_populates="session", cascade="all, delete-orphan"
79 )
80 proctor_events: Mapped[list[ProctorEvent]] = relationship(
81     back_populates="session", cascade="all, delete-orphan"
82 )
83 result: Mapped[Result | None] = relationship(
84     back_populates="session", cascade="all, delete-orphan", uselist=False
85 )
86
87 @property
88 def is_open(self) -> bool:
89     return self.status is SessionStatus.IN_PROGRESS
90
91
92 class Answer(UUIDPrimaryKeyMixin, TimestampMixin, Base):
93     __tablename__ = "answers"
94     __table_args__ = (UniqueConstraint("session_id", "question_id", name="uq_answer_per_question"),)
95
96     session_id: Mapped[uuid.UUID] = mapped_column(
97         PGUUID(as_uuid=True),
98         ForeignKey("exam_sessions.id", ondelete="CASCADE"),
99         nullable=False,
100         index=True,
101     )
102     question_id: Mapped[uuid.UUID] = mapped_column(
103         PGUUID(as_uuid=True), ForeignKey("questions.id", ondelete="RESTRICT"), nullable=False
104     )
105
106     # Objective answers: list of chosen option ids (stringified UUIDs).
107     selected_option_ids: Mapped[list[str] | None] = mapped_column(JSONB)
108     # Subjective answers: free text, or an object key for a handwritten scan.
109     text_answer: Mapped[str | None] = mapped_column(Text)
110     image_object_key: Mapped[str | None] = mapped_column(String(512))
111     # Server-generated so the examiner queue can render a grid of scans without pulling
112     # a dozen multi-megabyte originals through presigned URLs.
113     image_thumb_key: Mapped[str | None] = mapped_column(String(512))
114     # OCR pre-pass over a handwritten scan. Advisory only - the examiner reads the image.
115     ocr_text: Mapped[str | None] = mapped_column(Text)
116     ocr_confidence: Mapped[float | None] = mapped_column(Float)
117     # Recorded at save time so grading and analytics never re-tokenise the text.
118     word_count: Mapped[int | None] = mapped_column(Integer)
119
120     awarded_marks: Mapped[float | None] = mapped_column(Float)
121     max_marks: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
122     grade_status: Mapped[GradeStatus] = mapped_column(
123         Enum(GradeStatus, name="grade_status", values_callable=lambda e: [m.value for m in e]),
124         default=GradeStatus.UNANSWERED,
125         nullable=False,
126         index=True,
127     )
128     examiner_comment: Mapped[str | None] = mapped_column(Text)
129     graded_by_id: Mapped[uuid.UUID | None] = mapped_column(
130         PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
131     )
132     graded_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
133     answered_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
134
135     session: Mapped[ExamSession] = relationship(back_populates="answers")
136     question: Mapped[Question] = relationship()
137     graded_by: Mapped[User | None] = relationship(foreign_keys=[graded_by_id])
138     ai_evaluations: Mapped[list[AiEvaluation]] = relationship(
139         back_populates="answer",
140         cascade="all, delete-orphan",
141         order_by="AiEvaluation.created_at.desc()",
142     )
143
144 @property
145 def latest_ai_evaluation(self) -> AiEvaluation | None:
146     return self.ai_evaluations[0] if self.ai_evaluations else None
147
148 @property

```

```
149     def is_answered(self) -> bool:
150         return bool(self.selected_option_ids or self.text_answer or self.image_object_key)
```

models/proctor.py – ProctorEvent

backend/app/db/models/proctor.py · 56 lines

```
1  """Proctoring evidence: one row per behavioural or vision signal."""
2
3  from __future__ import annotations
4
5  import uuid
6  from datetime import datetime
7  from typing import TYPE_CHECKING, Any
8
9  from sqlalchemy import DateTime, Enum, Float, ForeignKey, Index, String
10 from sqlalchemy.dialects.postgresql import JSONB
11 from sqlalchemy.dialects.postgresql import UUID as PGUUID
12 from sqlalchemy.orm import Mapped, mapped_column, relationship
13
14 from app.db.base import Base, UUIDPrimaryKeyMixin
15 from app.db.models.enums import ProctorEventType, ProctorSeverity
16
17 if TYPE_CHECKING:
18     from app.db.models.exam_session import ExamSession
19
20
21 class ProctorEvent(UUIDPrimaryKeyMixin, Base):
22     __tablename__ = "proctor_events"
23     __table_args__ = (Index("ix_proctor_events_timeline", "session_id", "occurred_at"),)
24
25     session_id: Mapped[uuid.UUID] = mapped_column(
26         PGUUID(as_uuid=True),
27         ForeignKey("exam_sessions.id", ondelete="CASCADE"),
28         nullable=False,
29     )
30     event_type: Mapped[ProctorEventType] = mapped_column(
31         Enum(
32             ProctorEventType,
33             name="proctor_event_type",
34             values_callable=lambda e: [m.value for m in e],
35         ),
36         nullable=False,
37     )
38     severity: Mapped[ProctorSeverity] = mapped_column(
39         Enum(
40             ProctorSeverity,
41             name="proctor_severity",
42             values_callable=lambda e: [m.value for m in e],
43         ),
44         default=ProctorSeverity.INFO,
45         nullable=False,
46     )
47     # Client clock, echoed for ordering inside a batch; server_received_at is authoritative.
48     occurred_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
49     server_received_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
50     duration_ms: Mapped[int | None] = mapped_column()
51     weight: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
52     # e.g. {"face_count": 2, "yaw": -34.2, "confidence": 0.91}
53     event_metadata: Mapped[dict[str, Any] | None] = mapped_column("metadata", JSONB)
54     snapshot_object_key: Mapped[str | None] = mapped_column(String(512))
55
56     session: Mapped[ExamSession] = relationship(back_populates="proctor_events")
```

models/grading.py – AiEvaluation, Result

backend/app/db/models/grading.py · 67 lines

```
1  """Machine evaluations of subjective answers, and final per-session results."""
2
3  from __future__ import annotations
4
5  import uuid
6  from datetime import datetime
7  from typing import TYPE_CHECKING, Any
8
9  from sqlalchemy import Boolean, DateTime, Float, ForeignKey, Integer, String, Text
10 from sqlalchemy.dialects.postgresql import JSONB
11 from sqlalchemy.dialects.postgresql import UUID as PGUUID
12 from sqlalchemy.orm import Mapped, mapped_column, relationship
13
14 from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
15
16 if TYPE_CHECKING:
17     from app.db.models.exam_session import Answer, ExamSession
18
19
20 class AiEvaluation(UUIDPrimaryKeyMixin, TimestampMixin, Base):
21     """First-pass score produced by the configured grader.
22
23     Never authoritative on its own – an examiner reviews and may override it. Kept as a
24     history (one row per grading run) so a re-grade does not destroy the earlier verdict.
25     """
26
27     __tablename__ = "ai_evaluations"
28
29     answer_id: Mapped[uuid.UUID] = mapped_column(
30         PGUUID(as_uuid=True),
31         ForeignKey("answers.id", ondelete="CASCADE"),
32         nullable=False,
33         index=True,
34     )
35     provider: Mapped[str] = mapped_column(String(50), nullable=False)
36     model: Mapped[str | None] = mapped_column(String(100))
37     score: Mapped[float] = mapped_column(Float, nullable=False)
38     max_score: Mapped[float] = mapped_column(Float, nullable=False)
39     justification: Mapped[str] = mapped_column(Text, nullable=False)
40     confidence: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
41     raw_response: Mapped[dict[str, Any] | None] = mapped_column(JSONB)
42     error: Mapped[str | None] = mapped_column(Text)
43
44     answer: Mapped[Answer] = relationship(back_populates="ai_evaluations")
45
46
47 class Result(UUIDPrimaryKeyMixin, TimestampMixin, Base):
48     __tablename__ = "results"
49
50     session_id: Mapped[uuid.UUID] = mapped_column(
51         PGUUID(as_uuid=True),
52         ForeignKey("exam_sessions.id", ondelete="CASCADE"),
53         nullable=False,
54         unique=True,
55     )
56     total_marks: Mapped[float] = mapped_column(Float, nullable=False)
57     obtained_marks: Mapped[float] = mapped_column(Float, nullable=False)
58     percentage: Mapped[float] = mapped_column(Float, nullable=False)
59     correct_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
60     incorrect_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
61     unanswered_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
62     pending_review_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
63
64     published: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False, index=True)
65     published_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
66
67     session: Mapped[ExamSession] = relationship(back_populates="result")
```


III Alembic migrations

Linear history. Each revision carries both `upgrade()` and `downgrade()`.

03ad33e6bd09_initial_schema.py

backend/alembic/versions/03ad33e6bd09_initial_schema.py · 253 lines

```
1  """initial schema
2
3  Revision ID: 03ad33e6bd09
4  Revises:
5  Create Date: 2026-08-27 12:36:58.989165
6  """
7  from __future__ import annotations
8
9  from collections.abc import Sequence
10
11  import sqlalchemy as sa
12  from alembic import op
13  from sqlalchemy.dialects import postgresql
14  revision: str = '03ad33e6bd09'
15  down_revision: str | None = None
16  branch_labels: str | Sequence[str] | None = None
17  depends_on: str | Sequence[str] | None = None
18
19
20  def upgrade() -> None:
21      # ### commands auto generated by Alembic - please adjust! ###
22      op.create_table('subjects',
23          sa.Column('code', sa.String(length=32), nullable=False),
24          sa.Column('name', sa.String(length=150), nullable=False),
25          sa.Column('description', sa.Text(), nullable=True),
26          sa.Column('id', sa.UUID(), nullable=False),
27          sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
28          sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
29          sa.PrimaryKeyConstraint('id', name=op.f('pk_subjects'))
30      )
31      op.create_index(op.f('ix_subjects_code'), 'subjects', ['code'], unique=True)
32      op.create_table('users',
33          sa.Column('email', sa.String(length=255), nullable=False),
34          sa.Column('password_hash', sa.String(length=255), nullable=False),
35          sa.Column('full_name', sa.String(length=150), nullable=False),
36          sa.Column('role', sa.Enum('admin', 'examiner', 'candidate', name='user_role'), nullable=False),
37          sa.Column('is_active', sa.Boolean(), nullable=False),
38          sa.Column('access_status', sa.Enum('pending', 'approved', 'revoked', name='access_status'), nullable=False),
39          sa.Column('access_changed_at', sa.DateTime(timezone=True), nullable=True),
40          sa.Column('access_changed_by_id', sa.UUID(), nullable=True),
41          sa.Column('access_note', sa.Text(), nullable=True),
42          sa.Column('last_login_at', sa.DateTime(timezone=True), nullable=True),
43          sa.Column('id', sa.UUID(), nullable=False),
44          sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
45          sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
46          sa.ForeignKeyConstraint(['access_changed_by_id'], ['users.id'], name=op.f('fk_users_access_changed_by_id_users'),
ondelete='SET NULL'),
47          sa.PrimaryKeyConstraint('id', name=op.f('pk_users'))
48      )
49      op.create_index(op.f('ix_users_access_status'), 'users', ['access_status'], unique=False)
50      op.create_index(op.f('ix_users_email'), 'users', ['email'], unique=True)
51      op.create_index(op.f('ix_users_role'), 'users', ['role'], unique=False)
52      op.create_table('exams',
53          sa.Column('subject_id', sa.UUID(), nullable=False),
54          sa.Column('title', sa.String(length=200), nullable=False),
55          sa.Column('description', sa.Text(), nullable=True),
56          sa.Column('duration_minutes', sa.Integer(), nullable=False),
57          sa.Column('starts_at', sa.DateTime(timezone=True), nullable=False),
58          sa.Column('ends_at', sa.DateTime(timezone=True), nullable=False),
59          sa.Column('selection_rules', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
60          sa.Column('randomize', sa.Boolean(), nullable=False),
61          sa.Column('shuffle_options', sa.Boolean(), nullable=False),
62          sa.Column('negative_marking', sa.Boolean(), nullable=False),
63          sa.Column('paper_salt', sa.String(length=64), nullable=False),
64          sa.Column('proctor_config', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
65          sa.Column('grading_config', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
66          sa.Column('status', sa.Enum('draft', 'published', 'closed', name='exam_status'), nullable=False),
```

```

67     sa.Column('results_published', sa.Boolean(), nullable=False),
68     sa.Column('created_by_id', sa.UUID(), nullable=True),
69     sa.Column('id', sa.UUID(), nullable=False),
70     sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
71     sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
72     sa.ForeignKeyConstraint(['created_by_id'], ['users.id'], name=op.f('fk_exams_created_by_id_users'), ondelete='SET
NULL'),
73     sa.ForeignKeyConstraint(['subject_id'], ['subjects.id'], name=op.f('fk_exams_subject_id_subjects'),
ondelete='RESTRICT'),
74     sa.PrimaryKeyConstraint('id', name=op.f('pk_exams'))
75 )
76 op.create_index(op.f('ix_exams_status'), 'exams', ['status'], unique=False)
77 op.create_table('questions',
78     sa.Column('subject_id', sa.UUID(), nullable=False),
79     sa.Column('question_type', sa.Enum('mcq', 'multi_select', 'short_answer', 'long_answer', 'image_upload',
name='question_type'), nullable=False),
80     sa.Column('difficulty', sa.Enum('easy', 'medium', 'hard', name='difficulty'), nullable=False),
81     sa.Column('body', sa.Text(), nullable=False),
82     sa.Column('model_answer', sa.Text(), nullable=True),
83     sa.Column('rubric', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
84     sa.Column('marks', sa.Float(), nullable=False),
85     sa.Column('negative_marks', sa.Float(), nullable=False),
86     sa.Column('tags', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
87     sa.Column('is_active', sa.Boolean(), nullable=False),
88     sa.Column('created_by_id', sa.UUID(), nullable=True),
89     sa.Column('id', sa.UUID(), nullable=False),
90     sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
91     sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
92     sa.ForeignKeyConstraint(['created_by_id'], ['users.id'], name=op.f('fk_questions_created_by_id_users'),
ondelete='SET NULL'),
93     sa.ForeignKeyConstraint(['subject_id'], ['subjects.id'], name=op.f('fk_questions_subject_id_subjects'),
ondelete='RESTRICT'),
94     sa.PrimaryKeyConstraint('id', name=op.f('pk_questions'))
95 )
96 op.create_index(op.f('ix_questions_is_active'), 'questions', ['is_active'], unique=False)
97 op.create_index('ix_questions_pool_lookup', 'questions', ['subject_id', 'question_type', 'difficulty'],
unique=False)
98 op.create_table('exam_questions',
99     sa.Column('exam_id', sa.UUID(), nullable=False),
100     sa.Column('question_id', sa.UUID(), nullable=False),
101     sa.Column('marks_override', sa.Float(), nullable=True),
102     sa.Column('order_index', sa.Integer(), nullable=False),
103     sa.Column('id', sa.UUID(), nullable=False),
104     sa.ForeignKeyConstraint(['exam_id'], ['exams.id'], name=op.f('fk_exam_questions_exam_id_exams'),
ondelete='CASCADE'),
105     sa.ForeignKeyConstraint(['question_id'], ['questions.id'], name=op.f('fk_exam_questions_question_id_questions'),
ondelete='RESTRICT'),
106     sa.PrimaryKeyConstraint('id', name=op.f('pk_exam_questions')),
107     sa.UniqueConstraint('exam_id', 'question_id', name='uq_exam_question')
108 )
109 op.create_index(op.f('ix_exam_questions_exam_id'), 'exam_questions', ['exam_id'], unique=False)
110 op.create_table('exam_sessions',
111     sa.Column('exam_id', sa.UUID(), nullable=False),
112     sa.Column('candidate_id', sa.UUID(), nullable=False),
113     sa.Column('paper_seed', sa.String(length=64), nullable=False),
114     sa.Column('question_order', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
115     sa.Column('started_at', sa.DateTime(timezone=True), nullable=False),
116     sa.Column('expires_at', sa.DateTime(timezone=True), nullable=False),
117     sa.Column('submitted_at', sa.DateTime(timezone=True), nullable=True),
118     sa.Column('last_heartbeat_at', sa.DateTime(timezone=True), nullable=True),
119     sa.Column('status', sa.Enum('in_progress', 'submitted', 'auto_submitted', 'terminated', name='session_status'),
nullable=False),
120     sa.Column('suspicion_score', sa.Float(), nullable=False),
121     sa.Column('tab_switch_count', sa.Integer(), nullable=False),
122     sa.Column('is_flagged', sa.Boolean(), nullable=False),
123     sa.Column('termination_reason', sa.Text(), nullable=True),
124     sa.Column('id', sa.UUID(), nullable=False),
125     sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
126     sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
127     sa.ForeignKeyConstraint(['candidate_id'], ['users.id'], name=op.f('fk_exam_sessions_candidate_id_users'),
ondelete='CASCADE'),
128     sa.ForeignKeyConstraint(['exam_id'], ['exams.id'], name=op.f('fk_exam_sessions_exam_id_exams'),
ondelete='CASCADE'),
129     sa.PrimaryKeyConstraint('id', name=op.f('pk_exam_sessions')),
130     sa.UniqueConstraint('exam_id', 'candidate_id', name='uq_exam_session_attempt')
131 )
132 op.create_index(op.f('ix_exam_sessions_candidate_id'), 'exam_sessions', ['candidate_id'], unique=False)

```

```

133 op.create_index(op.f('ix_exam_sessions_exam_id'), 'exam_sessions', ['exam_id'], unique=False)
134 op.create_index(op.f('ix_exam_sessions_is_flagged'), 'exam_sessions', ['is_flagged'], unique=False)
135 op.create_index(op.f('ix_exam_sessions_status'), 'exam_sessions', ['status'], unique=False)
136 op.create_table('question_options',
137 sa.Column('question_id', sa.UUID(), nullable=False),
138 sa.Column('text', sa.Text(), nullable=False),
139 sa.Column('is_correct', sa.Boolean(), nullable=False),
140 sa.Column('order_index', sa.Integer(), nullable=False),
141 sa.Column('id', sa.UUID(), nullable=False),
142 sa.ForeignKeyConstraint(['question_id'], ['questions.id'], name=op.f('fk_question_options_question_id_questions'),
ondelete='CASCADE'),
143 sa.PrimaryKeyConstraint('id', name=op.f('pk_question_options'))
144 )
145 op.create_table('answers',
146 sa.Column('session_id', sa.UUID(), nullable=False),
147 sa.Column('question_id', sa.UUID(), nullable=False),
148 sa.Column('selected_option_ids', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
149 sa.Column('text_answer', sa.Text(), nullable=True),
150 sa.Column('image_object_key', sa.String(length=512), nullable=True),
151 sa.Column('awarded_marks', sa.Float(), nullable=True),
152 sa.Column('max_marks', sa.Float(), nullable=False),
153 sa.Column('grade_status', sa.Enum('unanswered', 'auto_scored', 'pending_ai', 'ai_scored', 'examiner_reviewed',
name='grade_status'), nullable=False),
154 sa.Column('examiner_comment', sa.Text(), nullable=True),
155 sa.Column('graded_by_id', sa.UUID(), nullable=True),
156 sa.Column('graded_at', sa.DateTime(timezone=True), nullable=True),
157 sa.Column('answered_at', sa.DateTime(timezone=True), nullable=True),
158 sa.Column('id', sa.UUID(), nullable=False),
159 sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
160 sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
161 sa.ForeignKeyConstraint(['graded_by_id'], ['users.id'], name=op.f('fk_answers_graded_by_id_users'), ondelete='SET
NULL'),
162 sa.ForeignKeyConstraint(['question_id'], ['questions.id'], name=op.f('fk_answers_question_id_questions'),
ondelete='RESTRICT'),
163 sa.ForeignKeyConstraint(['session_id'], ['exam_sessions.id'], name=op.f('fk_answers_session_id_exam_sessions'),
ondelete='CASCADE'),
164 sa.PrimaryKeyConstraint('id', name=op.f('pk_answers')),
165 sa.UniqueConstraint('session_id', 'question_id', name='uq_answer_per_question')
166 )
167 op.create_index(op.f('ix_answers_grade_status'), 'answers', ['grade_status'], unique=False)
168 op.create_index(op.f('ix_answers_session_id'), 'answers', ['session_id'], unique=False)
169 op.create_table('proctor_events',
170 sa.Column('session_id', sa.UUID(), nullable=False),
171 sa.Column('event_type', sa.Enum('face_missing', 'multiple_faces', 'gaze_away', 'tab_switch', 'window_blur',
'fullscreen_exit', 'camera_blocked', 'paste_attempt', 'copy_attempt', 'devtools_open', name='proctor_event_type'),
nullable=False),
172 sa.Column('severity', sa.Enum('info', 'warning', 'critical', name='proctor_severity'), nullable=False),
173 sa.Column('occurred_at', sa.DateTime(timezone=True), nullable=False),
174 sa.Column('server_received_at', sa.DateTime(timezone=True), nullable=False),
175 sa.Column('duration_ms', sa.Integer(), nullable=True),
176 sa.Column('weight', sa.Float(), nullable=False),
177 sa.Column('metadata', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
178 sa.Column('snapshot_object_key', sa.String(length=512), nullable=True),
179 sa.Column('id', sa.UUID(), nullable=False),
180 sa.ForeignKeyConstraint(['session_id'], ['exam_sessions.id'],
name=op.f('fk_proctor_events_session_id_exam_sessions'), ondelete='CASCADE'),
181 sa.PrimaryKeyConstraint('id', name=op.f('pk_proctor_events'))
182 )
183 op.create_index('ix_proctor_events_timeline', 'proctor_events', ['session_id', 'occurred_at'], unique=False)
184 op.create_table('results',
185 sa.Column('session_id', sa.UUID(), nullable=False),
186 sa.Column('total_marks', sa.Float(), nullable=False),
187 sa.Column('obtained_marks', sa.Float(), nullable=False),
188 sa.Column('percentage', sa.Float(), nullable=False),
189 sa.Column('correct_count', sa.Integer(), nullable=False),
190 sa.Column('incorrect_count', sa.Integer(), nullable=False),
191 sa.Column('unanswered_count', sa.Integer(), nullable=False),
192 sa.Column('pending_review_count', sa.Integer(), nullable=False),
193 sa.Column('published', sa.Boolean(), nullable=False),
194 sa.Column('published_at', sa.DateTime(timezone=True), nullable=True),
195 sa.Column('id', sa.UUID(), nullable=False),
196 sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
197 sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
198 sa.ForeignKeyConstraint(['session_id'], ['exam_sessions.id'], name=op.f('fk_results_session_id_exam_sessions'),
ondelete='CASCADE'),
199 sa.PrimaryKeyConstraint('id', name=op.f('pk_results')),
200 sa.UniqueConstraint('session_id', name=op.f('uq_results_session_id'))

```

```

201     )
202     op.create_index(op.f('ix_results_published'), 'results', ['published'], unique=False)
203     op.create_table('ai_evaluations',
204         sa.Column('answer_id', sa.UUID(), nullable=False),
205         sa.Column('provider', sa.String(length=50), nullable=False),
206         sa.Column('model', sa.String(length=100), nullable=True),
207         sa.Column('score', sa.Float(), nullable=False),
208         sa.Column('max_score', sa.Float(), nullable=False),
209         sa.Column('justification', sa.Text(), nullable=False),
210         sa.Column('confidence', sa.Float(), nullable=False),
211         sa.Column('raw_response', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
212         sa.Column('error', sa.Text(), nullable=True),
213         sa.Column('id', sa.UUID(), nullable=False),
214         sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
215         sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
216         sa.ForeignKeyConstraint(['answer_id'], ['answers.id'], name=op.f('fk_ai_evaluations_answer_id_answers'),
ondelete='CASCADE'),
217         sa.PrimaryKeyConstraint('id', name=op.f('pk_ai_evaluations'))
218     )
219     op.create_index(op.f('ix_ai_evaluations_answer_id'), 'ai_evaluations', ['answer_id'], unique=False)
220     ### end Alembic commands ###
221
222
223     def downgrade() -> None:
224         ### commands auto generated by Alembic - please adjust! ###
225         op.drop_index(op.f('ix_ai_evaluations_answer_id'), table_name='ai_evaluations')
226         op.drop_table('ai_evaluations')
227         op.drop_index(op.f('ix_results_published'), table_name='results')
228         op.drop_table('results')
229         op.drop_index('ix_proctor_events_timeline', table_name='proctor_events')
230         op.drop_table('proctor_events')
231         op.drop_index(op.f('ix_answers_session_id'), table_name='answers')
232         op.drop_index(op.f('ix_answers_grade_status'), table_name='answers')
233         op.drop_table('answers')
234         op.drop_table('question_options')
235         op.drop_index(op.f('ix_exam_sessions_status'), table_name='exam_sessions')
236         op.drop_index(op.f('ix_exam_sessions_is_flagged'), table_name='exam_sessions')
237         op.drop_index(op.f('ix_exam_sessions_exam_id'), table_name='exam_sessions')
238         op.drop_index(op.f('ix_exam_sessions_candidate_id'), table_name='exam_sessions')
239         op.drop_table('exam_sessions')
240         op.drop_index(op.f('ix_exam_questions_exam_id'), table_name='exam_questions')
241         op.drop_table('exam_questions')
242         op.drop_index('ix_questions_pool_lookup', table_name='questions')
243         op.drop_index(op.f('ix_questions_is_active'), table_name='questions')
244         op.drop_table('questions')
245         op.drop_index(op.f('ix_exams_status'), table_name='exams')
246         op.drop_table('exams')
247         op.drop_index(op.f('ix_users_role'), table_name='users')
248         op.drop_index(op.f('ix_users_email'), table_name='users')
249         op.drop_index(op.f('ix_users_access_status'), table_name='users')
250         op.drop_table('users')
251         op.drop_index(op.f('ix_subjects_code'), table_name='subjects')
252         op.drop_table('subjects')
253         ### end Alembic commands ###

```

a17b4c9de204_word_bounds_thumbnails_ocr.py

backend/alembic/versions/a17b4c9de204_word_bounds_thumbnails_ocr.py · 37 lines

```
1  """answer word bounds, image thumbnails, OCR pre-pass
2
3  Revision ID: a17b4c9de204
4  Revises: f0c6eeb2272c
5  Create Date: 2026-09-01 10:12:03.881204
6  """
7  from __future__ import annotations
8
9  from collections.abc import Sequence
10
11  import sqlalchemy as sa
12  from alembic import op
13
14  revision: str = 'a17b4c9de204'
15  down_revision: str | None = 'f0c6eeb2272c'
16  branch_labels: str | Sequence[str] | None = None
17  depends_on: str | Sequence[str] | None = None
18
19
20  def upgrade() -> None:
21      op.add_column('questions', sa.Column('min_words', sa.Integer(), nullable=True))
22      op.add_column('questions', sa.Column('max_words', sa.Integer(), nullable=True))
23
24      op.add_column('answers', sa.Column('image_thumb_key', sa.String(length=512), nullable=True))
25      op.add_column('answers', sa.Column('ocr_text', sa.Text(), nullable=True))
26      op.add_column('answers', sa.Column('ocr_confidence', sa.Float(), nullable=True))
27      op.add_column('answers', sa.Column('word_count', sa.Integer(), nullable=True))
28
29
30  def downgrade() -> None:
31      op.drop_column('answers', 'word_count')
32      op.drop_column('answers', 'ocr_confidence')
33      op.drop_column('answers', 'ocr_text')
34      op.drop_column('answers', 'image_thumb_key')
35
36      op.drop_column('questions', 'max_words')
37      op.drop_column('questions', 'min_words')
```


f0c6eeb2272c_login_access_requests_exam_enrollments_.py

backend/alembic/versions/f0c6eeb2272c_login_access_requests_exam_enrollments_.py · 116 lines

```
1  """login access requests, exam enrollments, split names
2
3  Revision ID: f0c6eeb2272c
4  Revises: 03ad33e6bd09
5  Create Date: 2026-08-27 16:44:14.186364
6  """
7  from __future__ import annotations
8
9  from collections.abc import Sequence
10
11  import sqlalchemy as sa
12  from alembic import op
13
14  revision: str = 'f0c6eeb2272c'
15  down_revision: str | None = '03ad33e6bd09'
16  branch_labels: str | Sequence[str] | None = None
17  depends_on: str | Sequence[str] | None = None
18
19
20  def upgrade() -> None:
21      # ### commands auto generated by Alembic - please adjust! ###
22      op.create_table('login_access_requests',
23                      sa.Column('candidate_id', sa.UUID(), nullable=False),
24                      sa.Column('status', sa.Enum('pending', 'approved', 'rejected', name='login_access_status'), nullable=False),
25                      sa.Column('requested_at', sa.DateTime(timezone=True), nullable=False),
26                      sa.Column('reviewed_at', sa.DateTime(timezone=True), nullable=True),
27                      sa.Column('reviewed_by_id', sa.UUID(), nullable=True),
28                      sa.Column('review_note', sa.Text(), nullable=True),
29                      sa.Column('id', sa.UUID(), nullable=False),
30                      sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
31                      sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
32                      sa.ForeignKeyConstraint(['candidate_id'], ['users.id'], name=op.f('fk_login_access_requests_candidate_id_users'),
ondelete='CASCADE'),
33                      sa.ForeignKeyConstraint(['reviewed_by_id'], ['users.id'],
name=op.f('fk_login_access_requests_reviewed_by_id_users'), ondelete='SET NULL'),
34                      sa.PrimaryKeyConstraint('id', name=op.f('pk_login_access_requests'))
35      )
36      op.create_index(op.f('ix_login_access_requests_candidate_id'), 'login_access_requests', ['candidate_id'],
unique=True)
37      op.create_index(op.f('ix_login_access_requests_status'), 'login_access_requests', ['status'], unique=False)
38      op.create_table('exam_enrollments',
39                      sa.Column('exam_id', sa.UUID(), nullable=False),
40                      sa.Column('candidate_id', sa.UUID(), nullable=False),
41                      sa.Column('assigned_by_id', sa.UUID(), nullable=True),
42                      sa.Column('assigned_at', sa.DateTime(timezone=True), nullable=False),
43                      sa.Column('id', sa.UUID(), nullable=False),
44                      sa.ForeignKeyConstraint(['assigned_by_id'], ['users.id'], name=op.f('fk_exam_enrollments_assigned_by_id_users'),
ondelete='SET NULL'),
45                      sa.ForeignKeyConstraint(['candidate_id'], ['users.id'], name=op.f('fk_exam_enrollments_candidate_id_users'),
ondelete='CASCADE'),
46                      sa.ForeignKeyConstraint(['exam_id'], ['exams.id'], name=op.f('fk_exam_enrollments_exam_id_exams'),
ondelete='CASCADE'),
47                      sa.PrimaryKeyConstraint('id', name=op.f('pk_exam_enrollments')),
48                      sa.UniqueConstraint('exam_id', 'candidate_id', name='uq_exam_enrollment')
49      )
50      op.create_index(op.f('ix_exam_enrollments_candidate_id'), 'exam_enrollments', ['candidate_id'], unique=False)
51      op.create_index(op.f('ix_exam_enrollments_exam_id'), 'exam_enrollments', ['exam_id'], unique=False)
52      # Added with a server default so the columns can be NOT NULL on a populated table;
53      # the default is dropped again once the backfill below has run.
54      op.add_column(
55          'users',
56          sa.Column('first_name', sa.String(length=80), nullable=False, server_default=''),
57      )
58      op.add_column(
59          'users',
60          sa.Column('last_name', sa.String(length=80), nullable=False, server_default=''),
61      )
62      # ### end Alembic commands ###
63
64      # --- data migration -----
65      # Split existing full names: everything before the first space is the first name,
66      # the remainder is the last name. Nothing is deleted and full_name is left intact.
```

```

67     op.execute(
68         """
69         UPDATE users
70             SET first_name = split_part(trim(full_name), ' ', 1),
71                 last_name = trim(substr(trim(full_name),
72                                     length(split_part(trim(full_name), ' ', 1)) + 1))
73             WHERE first_name = ''
74         """
75     )
76     op.alter_column('users', 'first_name', server_default=None)
77     op.alter_column('users', 'last_name', server_default=None)
78
79     # Existing candidates were already using the platform, so grandfather them in as
80     # approved rather than locking everyone out behind a new pending gate.
81     op.execute(
82         """
83         INSERT INTO login_access_requests
84             (id, candidate_id, status, requested_at, reviewed_at, review_note,
85              created_at, updated_at)
86         SELECT gen_random_uuid(), u.id, 'approved', u.created_at, now(),
87                'Granted automatically when login approval was introduced', now(), now()
88         FROM users u
89         WHERE u.role = 'candidate'
90         """
91     )
92
93     # Likewise, candidates could previously see every published exam. Enrol them in the
94     # exams that already exist so nobody loses access to a paper they could already sit.
95     op.execute(
96         """
97         INSERT INTO exam_enrollments (id, exam_id, candidate_id, assigned_at)
98         SELECT gen_random_uuid(), e.id, u.id, now()
99         FROM exams e
100        CROSS JOIN users u
101        WHERE u.role = 'candidate'
102        """
103     )
104
105
106 def downgrade() -> None:
107     # ### commands auto generated by Alembic - please adjust! ###
108     op.drop_column('users', 'last_name')
109     op.drop_column('users', 'first_name')
110     op.drop_index(op.f('ix_exam_enrollments_exam_id'), table_name='exam_enrollments')
111     op.drop_index(op.f('ix_exam_enrollments_candidate_id'), table_name='exam_enrollments')
112     op.drop_table('exam_enrollments')
113     op.drop_index(op.f('ix_login_access_requests_status'), table_name='login_access_requests')
114     op.drop_index(op.f('ix_login_access_requests_candidate_id'), table_name='login_access_requests')
115     op.drop_table('login_access_requests')
116     # ### end Alembic commands ###

```